

ISTE 430 - Information Requirements Modeling

Spring 2026

Final Assessment: Group Project

Group 8

Members

Blen Nima	754002486
Marilyn Igwe	759004176

Due Date: May 3rd, 11:59 pm

Manifesto

We both equally contributed to the work.

Blen Nima

- Contributed to system planning and project scope definition
- Participated in requirement gathering and interview design
- Assisted in creating Context, Level-0, and Level-1 DFDs
- Contributed to abstract and detailed ERD development
- Helped design user interfaces (forms and reports)
- Participated in logical modeling and normalization (up to 3NF)
- Assisted in SQL database implementation and table creation
- Contributed to Oracle APEX application development
- Participated in testing and validation of system functionality
- Contributed to report writing and final documentation

Marilyn Igwe

- Contributed to system planning and project scope definition
- Participated in requirement gathering and interview design
- Assisted in creating Context, Level-0, and Level-1 DFDs
- Contributed to abstract and detailed ERD development
- Helped design user interfaces (forms and reports)
- Participated in logical modeling and normalization (up to 3NF)
- Assisted in SQL database implementation and table creation
- Contributed to Oracle APEX application development
- Participated in testing and validation of system functionality
- Contributed to report writing and final documentation

Table Of Contents

Manifesto.....	1
Table Of Contents.....	2
1. Introduction.....	4
1.1 Organization Background.....	4
1.2 Current Problems in Hospital Data Management.....	4
1.3 Proposed Hospital Database System.....	4
2. Phase 1: Systems Planning and Selection.....	5
2.1 Need for the New System.....	5
2.2 Project Scope.....	5
2.3 Baseline Project Plan.....	5
2.3.1 System Description.....	5
2.3.2 Alternatives Considered.....	5
2.3.3 Expected Benefits.....	6
2.3.4 Resource Requirements.....	6
2.3.5 Project Schedule.....	6
2.3.6 Estimated Costs.....	6
2.3.7 Project Risks.....	7
2.4 Feasibility Assessment.....	7
2.4.1 Technical Feasibility.....	7
2.4.2 Economic Feasibility.....	7
2.4.3 Organizational Feasibility.....	7
2.5 Risk Analysis and Mitigation.....	7
3. Phase 2: Systems Analysis.....	8
3.1 Requirement Determination.....	8
3.2 Interview Participants.....	8
3.3 Interview Questions and Question Types.....	9
3.4 Formal Interview Guides.....	11
3.5 Requirement Structuring Overview.....	12
3.6 Context DFD.....	12
3.7 Level-0 DFD.....	13
3.8 Level-n DFDs.....	15
3.9 Decision Tables.....	16
3.9.1 Decision Table 1: Appointment Scheduling.....	16
3.9.1.1 Unsimplified Decision Table.....	16
3.9.1.2 Appointment Scheduling Decision Table Reduction.....	17
3.9.1.3 Simplified Decision Table.....	17
3.9.2 Decision Table 2: Billing and Payment Processing.....	18
3.9.2.1 Unsimplified Decision Table.....	18
3.9.2.2 Billing and Payment Decision Table Reduction.....	18
3.9.2.3 Simplified Decision Table.....	19
3.10 Abstract ERD.....	20

4. Phase 3: System Design.....	20
4.1 Designing the Human Interface.....	20
4.1.1 Patient Registration Form.....	21
4.1.2 Appointment Scheduling Form.....	22
4.1.3 Doctor Consultation Form.....	23
4.1.4 Laboratory Results Report.....	25
4.2 Interface Dialogue Design.....	26
4.2.1 Dialogue Diagram.....	26
4.3 Anticipating and Handling Input Errors.....	26
4.4 Logical Design.....	27
4.4.1 Entity and Attribute Identification.....	27
4.4.2 Relationship Cardinalities and Labels.....	30
4.4.3 Relational Model / Shorthand Relations.....	31
4.4.4 Normalization to 3NF.....	32
Step 1: First Normal Form (1NF).....	32
Step 2: Second Normal Form (2NF).....	32
Step 3: Third Normal Form (3NF).....	32
4.5.1 DBMS Selection.....	33
4.5.2 Table Creation and Data Types.....	33
4.5.3 Constraints and Relationships.....	34
4.5.4 Indexes.....	34
4.5.5 Sample Data and Implementation.....	35
4.5.6 System Demonstration.....	35
5. Phase 4: Web Application Implementation.....	37
5.1 Application Overview.....	37
5.2 Login Interface.....	37
5.3 Dashboard Interface.....	37
5.4 Patient Registration Form.....	38
5.5 Appointment Scheduling Form.....	38
5.6 Billing Form.....	39
5.7 Billing Summary Report.....	39
5.8 Database Connectivity.....	39
6. Conclusion.....	40
7. References.....	41

1. Introduction

1.1 Organization Background

The healthcare industry depends heavily on accurate and secure data to ensure effective patient care. In a hospital environment, large volumes of information are handled daily, including patient records, appointments, laboratory results, prescriptions, and billing details.

The organization considered in this project is a mid-sized general hospital that provides both outpatient and inpatient services. It manages multiple departments, including laboratory testing, pharmacy services, and administrative billing, serving a large number of patients daily across multiple departments.

1.2 Current Problems in Hospital Data Management

Currently, the hospital faces several operational challenges due to the absence of a centralized data management system. Many processes still rely on paper-based records and disconnected digital tools, which slows down workflows and increase the likelihood of errors.

For example, retrieving patient records during emergencies can take significant time, delaying treatment. In addition, inconsistent data across departments often results in duplicate or mismatched patient information.

Appointment scheduling is also inefficient, as there is no real-time coordination between departments, leading to overlapping bookings and long waiting times. Furthermore, weak data security practices increase the risk of unauthorized access to sensitive patient information.

1.3 Proposed Hospital Database System

To address these issues, a centralized Hospital Database Management System is proposed. This system will integrate patient records, doctor schedules, appointments, laboratory reports, billing information, and staff data into a single platform.

With this system, receptionists can manage appointments electronically, doctors can access patient histories instantly, and administrative staff can generate accurate billing records. This improves coordination between departments and reduces delays in service delivery.

In addition, role-based access control ensures that only authorized personnel can view or modify sensitive data, improving overall data security.

Overall, the system will make hospital operations faster and more organized while allowing staff to access accurate patient information when it is needed most.

2. Phase 1: Systems Planning and Selection

2.1 Need for the New System

The hospital's current reliance on manual records and disconnected systems creates several inefficiencies in daily operations. Staff members spend unnecessary time searching for patient records, which can delay treatment, especially in urgent situations.

In addition, the lack of coordination between departments leads to scheduling conflicts, duplicate records, and inconsistent data. These issues not only reduce efficiency but also affect the quality of patient care.

Therefore, a centralized database system is necessary to streamline operations, improve data accuracy, and ensure secure and timely access to information across all departments.

2.2 Project Scope

The proposed Hospital Database Management System will provide a centralized platform to manage patient information and support core hospital operations. It will integrate multiple departments into a single database system to improve communication and coordination.

Key functionalities include managing patient records, doctor and department information, appointment scheduling, medical histories, laboratory results, prescriptions, billing, and staff data.

The system will be accessible to authorized users such as receptionists, doctors, nurses, lab technicians, pharmacists, and administrative staff. Access to different features will be controlled based on user roles to ensure data security.

However, the system will focus only on internal hospital operations. Advanced features such as telemedicine, mobile applications, external integrations, and AI-based diagnostics are outside the scope of this project and may be considered in future development.

2.3 Baseline Project Plan

2.3.1 System Description

The proposed system replaces the hospital's existing paper-based and fragmented record-keeping methods with a centralized database. Storing all data in a single system, it allows staff to access and update information in real time.

This allows quicker response time while retrieving records, allows departments to share information more easily, and reduces the time staff spend on manual administrative tasks.

2.3.2 Alternatives Considered

Three alternatives were evaluated before selecting the proposed system.

Alternative 1: Continue using the current manual system

This option requires no implementation cost but fails to address key issues such as inefficiency, data redundancy, slow record retrieval, poor coordination, and security risks.

Alternative 2: Purchase an off-the-shelf system

Commercial systems offer advanced features but are expensive and may include unnecessary functionalities that do not align with the hospital's needs. Customization can further increase costs.

Alternative 3: Develop a custom system (Selected Option)

This option provides a tailored solution that meets the hospital's specific requirements while remaining cost-effective and flexible for future expansion.

The custom system was selected due to its flexibility, cost-effectiveness, and alignment with hospital requirements.

2.3.3 Expected Benefits

- **Tangible Benefits:** Reduced paperwork, faster billing processes, fewer data entry errors, and lower administrative workload.
- **Intangible Benefits:** Improved communication between departments, increased staff productivity, better patient experience, and stronger data security.

2.3.4 Resource Requirements

- The project requires human resources such as a system analyst, database developer, and IT support staff.
- Software requirements include a DBMS Oracle APEX & Oracle Database, MySQL or PostgreSQL, SQL tools, and diagramming software for system design.
- The hospital's existing hardware infrastructure will be used, with the addition of a dedicated server if needed.
- The estimated time required to complete the project is approximately four weeks.

2.3.5 Project Schedule

The project will follow a structured timeline aligned with the SDLC:

- Week 1: Planning and selection
- Week 2: Systems analysis
- Week 3: Systems design
- Week 4: Implementation and testing

2.3.6 Estimated Costs

- **One-time costs:** System development, staff training, data migration, and initial setup.
- **Recurring costs:** System maintenance, technical support, data storage, and periodic updates.

2.3.7 Project Risks

- Staff resistance to change → addressed through training and awareness
- Data migration errors → mitigated through validation and verification
- System downtime or technical failures → minimized through backups and maintenance
- Security risks → controlled through authentication and role-based access

By identifying and managing these risks early, the project increases its likelihood of successful implementation.

2.4 Feasibility Assessment

2.4.1 Technical Feasibility

The system is technically feasible as it can be implemented using widely used database technologies such as MySQL or PostgreSQL. These tools are reliable and well-supported, making them suitable for this project.

The hospital already has basic infrastructure such as computers and network connectivity, which reduces the need for additional setup.

2.4.2 Economic Feasibility

The initial cost includes development, system setup, data migration, and staff training. Since open-source technologies can be used, the overall cost remains relatively low.

Over time, the system will help lower operational costs by reducing paperwork, minimizing errors, and saving staff time during daily tasks.

2.4.3 Organizational Feasibility

The system aligns with the hospital's goal of improving service quality and making daily operations more efficient. While some staff may initially resist the change, proper training and a user-friendly interface can help ensure smooth adoption.

2.5 Risk Analysis and Mitigation

Risk	Likelihood	Impact	Mitigation Strategy
Data migration errors	High	High	Validate and verify data during migration; perform sample checks
Unauthorized access to patient data	High	High	Implement authentication and role-based access control

Staff Resistance to new system	Medium	Medium	Provide training and demonstrate system benefits
System downtime or failure	Medium	Medium	Perform regular backups and maintenance
Data loss	Low	High	Use backup and recovery mechanisms
Poor usability/ UI issues	Low	Medium	Usability testing

The most critical risks in this project are data migration errors and security vulnerabilities, as they directly affect data integrity and patient confidentiality. These risks are mitigated through validation procedures, controlled access mechanisms, and regular system backups.

Operational risks such as staff resistance are addressed through training and user-friendly system design. Technical risks are minimized through proper system maintenance and testing.

This analysis highlights the most critical risks and ensures that appropriate measures are in place to reduce their impact during system implementation.

3. Phase 2: Systems Analysis

3.1 Requirement Determination

Requirement determination focuses on identifying the information needs, system functions, and user requirements of the proposed Hospital Database Management System. Since this project is based on a proposed hospital scenario, representative stakeholders were selected to reflect the main users of the system. The interview guides were designed to simulate the type of requirements that would normally be collected from real hospital staff.

In addition to interviews, contextual inquiry was used to understand how tasks such as patient registration, appointment scheduling, laboratory reporting, prescription handling, and billing are performed within the hospital environment. Relevant documents such as registration forms and billing records were also reviewed to identify required data fields and processing steps.

The collected requirements support the development of the DFDs, decision tables, and conceptual ERD presented in the following sections.

3.2 Interview Participants

Participant	Reason for Selection
Receptionist	Handles patient registration, appointment booking, and patient check-in.

Doctor	Uses patient history, diagnoses, treatments, and lab results.
Laboratory Technician	Manages lab test requests and records test results.
Pharmacist	Reviews prescriptions and manages medication dispensing.
Billing Officer	Handles invoices, payments, and billing records.
Hospital Administrator	Reviews reports and monitors hospital operations.

3.3 Interview Questions and Question Types

To determine system requirements for the proposed Hospital Database Management System, structured interviews were conducted with representative stakeholders, including receptionists, doctors, laboratory technicians, pharmacists, billing officers, and hospital administrators. These interviews included both open-ended questions (to gather detailed workflow insights) and closed-ended questions (to collect specific operational requirements).

Receptionist Interview Questions

Open-ended questions

How do you currently register new patients?

What challenges do you face while scheduling appointments?

How do you check doctor availability before confirming appointments?

What information do patients usually provide during registration?

Closed-ended questions

Do you need access to patient appointment history? (Yes/No)

Should the system automatically detect scheduling conflicts? (Yes/No)

Should appointment reminders be generated automatically? (Yes/No)

Justification: Open-ended questions help identify workflow steps and challenges in patient registration, while closed-ended questions confirm required system features such as automated scheduling support.

Doctor Interview Questions

Open-ended questions

What patient information do you need before starting consultation?

How do you currently review laboratory test results?

What difficulties do you face when accessing patient history?

Closed-ended questions

Should doctors be able to update medical records electronically? (Yes/No)

Should prescriptions be stored digitally in the system? (Yes/No)

Should doctors access lab results directly from the database? (Yes/No)

Justification: Open-ended questions identify clinical workflow needs, while closed-ended questions confirm functional requirements for electronic records and prescriptions.

Laboratory Technician Interview Questions

Open-ended questions

How are lab test requests received from doctors?
How are test results currently recorded and shared?
What challenges exist in tracking pending lab tests?

Closed-ended questions

Should lab test requests be generated electronically? (Yes/No)
Should technicians upload results directly to the system? (Yes/No)
Should doctors receive automatic notifications when results are ready? (Yes/No)

Justification: These questions determine requirements for digital lab test processing and result tracking.

Pharmacist Interview Questions

Open-ended questions

How do you receive prescriptions from doctors?
How do you check medication availability?
What issues occur when dispensing medication?

Closed-ended questions

Should prescriptions be stored electronically? (Yes/No)
Should the system track medication inventory levels? (Yes/No)
Should alerts appear for unavailable medications? (Yes/No)

Justification: Questions identify requirements for prescription processing and medication inventory tracking.

Billing Officer Interview Questions

Open-ended questions

How are patient bills currently generated?
What billing errors occur most frequently?
How do you track completed payments?

Closed-ended questions

Should billing be generated automatically after appointments? (Yes/No)
Should the system support multiple payment methods? (Yes/No)
Should unpaid invoices be tracked automatically? (Yes/No)

Justification: These questions identify requirements for automated billing and payment tracking.

Hospital Administrator Interview Questions

Open-ended questions

What operational reports are required for management decisions?
What challenges exist in monitoring hospital performance?
What improvements are expected from the new system?

Closed-ended questions

Should the system generate monthly performance reports? (Yes/No)
Should administrators monitor staff activity through reports? (Yes/No)
Should access permissions be role-based? (Yes/No)

Justification: These questions determine reporting and administrative control requirements.

3.4 Formal Interview Guides

Formal interview guides were prepared to ensure consistency and structure during requirement gathering from hospital stakeholders. Each interview followed a standardized format including interview purpose, participant role, key discussion areas, and expected outcomes. This approach ensured that requirements related to patient management, appointments, laboratory processing, prescriptions, billing, and reporting were captured systematically.

Interviewee	Purpose	Discussion Areas	Expected Outcomes
Receptionist	Understand patient registration and appointment scheduling processes	Patient registration steps; appointment booking procedures; checking doctor availability; scheduling challenges	Electronic patient registration; automated appointment scheduling; conflict detection; real-time doctor availability
Doctor	Understand access to patient medical records and treatment documentation	Accessing patient history; recording diagnoses and treatment notes; issuing prescriptions; reviewing lab results	Electronic medical records; digital prescription storage; real-time lab result access; centralized patient history tracking
Laboratory Technician	Understand lab test request handling and reporting workflows	Receiving test requests; recording lab results; communicating results to doctors; tracking pending tests	Electronic lab test submission; direct result entry; automatic doctor notifications; lab request status tracking
Pharmacist	Understand prescription processing and medication inventory tracking	Receiving prescriptions; verifying medication availability; dispensing medicines; monitoring inventory	Digital prescription verification; medication availability tracking; inventory monitoring; alerts for unavailable medications
Billing Officer	Understand billing generation and payment tracking processes	Generate invoices; recording payments; managing outstanding balances; correcting billing errors	Automated invoice generation; integrated payment tracking; support for multiple payment methods; monitoring unpaid bills

Hospital Administrator	Understand reporting and monitoring requirements	Required management reports; monitoring hospital performance; managing access permissions; identifying operational issues	Automated management reports; role-based access control; performance monitoring dashboards; improved operational oversight
------------------------	--	---	--

3.5 Requirement Structuring Overview

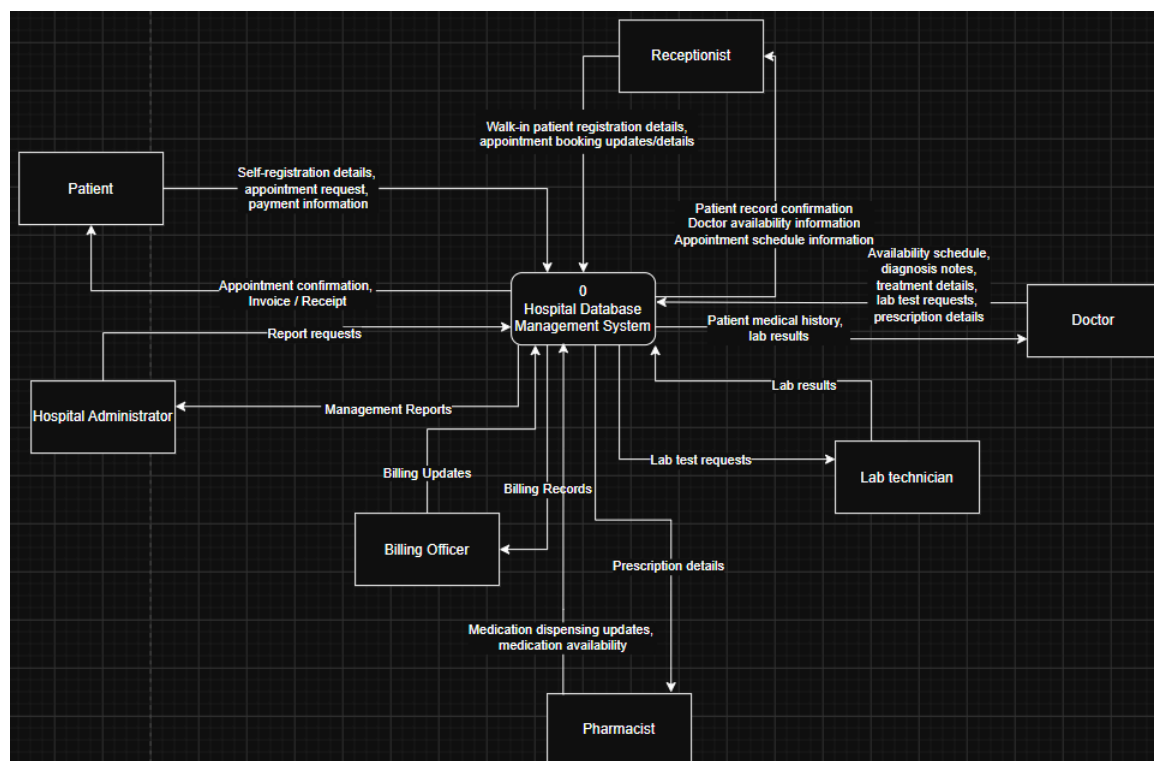
Requirement structuring organizes the information collected during requirement determination into models that show how data moves through the proposed Hospital Database Management System. The main techniques used in this section are Data Flow Diagrams, decision tables, and an abstract Entity-Relationship Diagram.

The DFDs show how external users interact with the system and how information flows between major processes such as patient registration, appointment scheduling, medical record management, laboratory testing, pharmacy processing, and billing. Decision tables are used to model business rules for selected processes. The abstract ERD identifies the major data entities and relationships required for the hospital database.

3.6 Context DFD

The Context DFD represents the Hospital Database Management System as one complete process. It shows the main external entities that interact with the system and the major data flows between them.

Main Process: Hospital Database Management System



Link:

<https://app.diagrams.net/#G14uvimZFreidc1c4tzAy9MXiiID1XbfYt#%7B%22pageId%22%3A%22UvkBA89y58eMA4gJKTpb%22%7D>

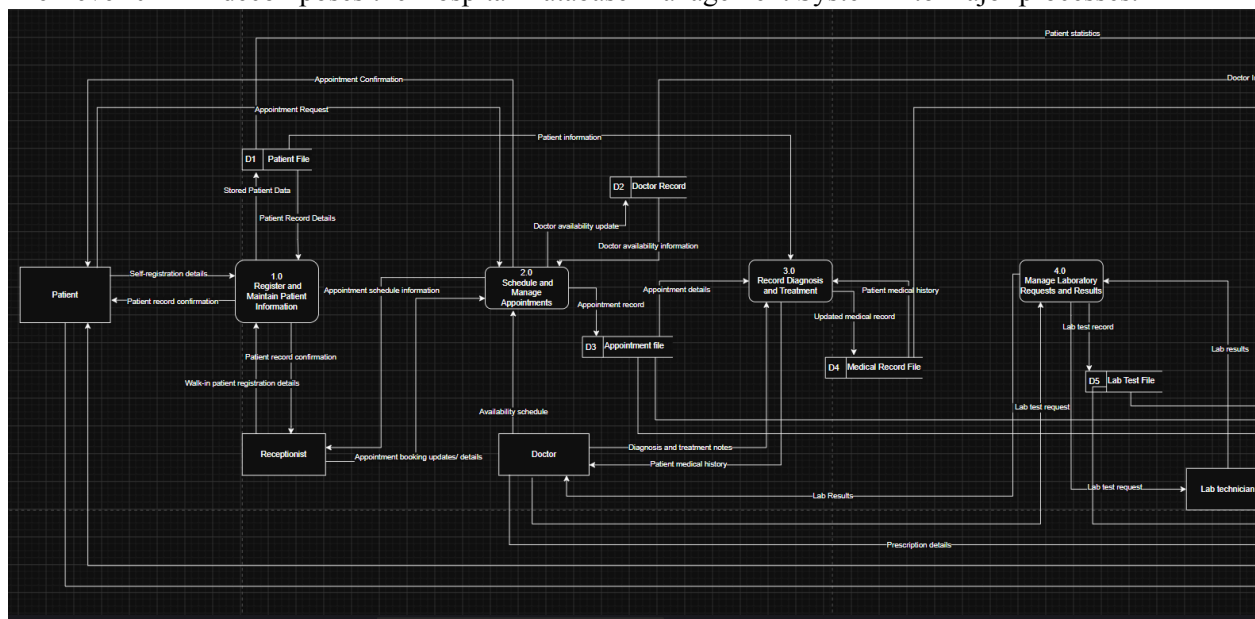
At the context level, the Hospital Database Management System receives information from hospital users and returns processed information to support daily operations. Patients provide self-registration details, appointment requests, and payment information, while hospital staff enter and retrieve operational data as part of their responsibilities. Receptionists manage walk-in patient registration and appointment scheduling activities through the system.

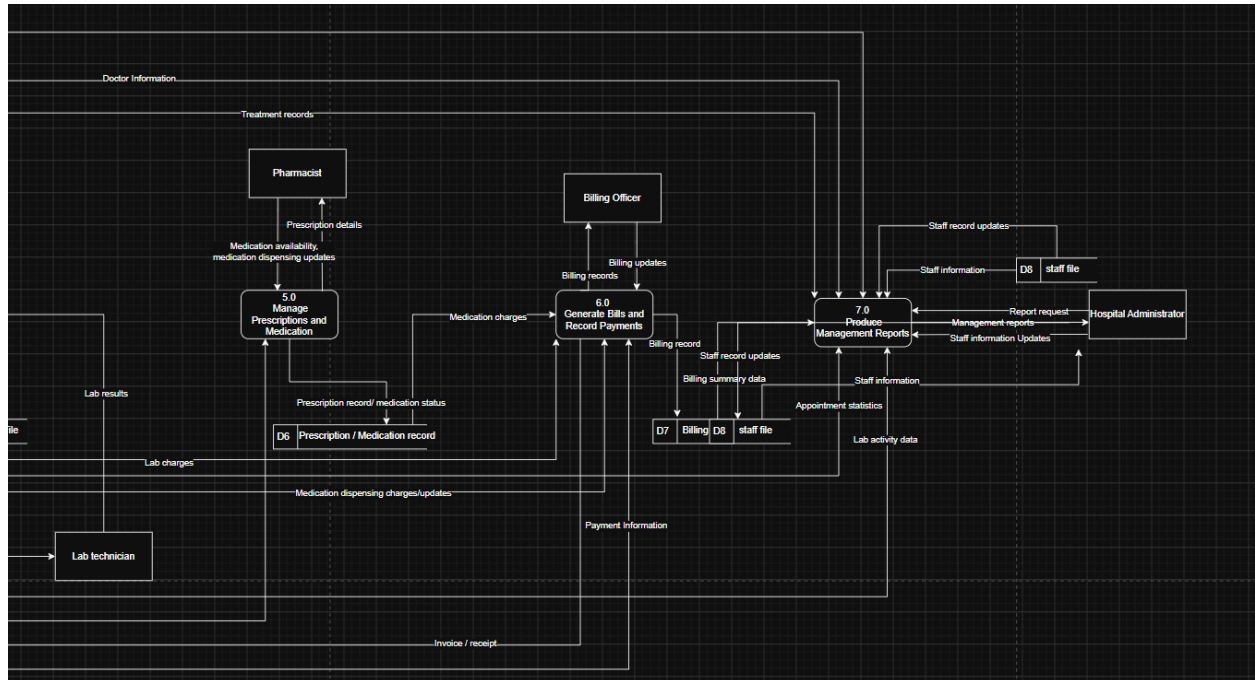
Doctors use the system to access patient medical histories, submit diagnoses and treatment details, request laboratory tests, and create prescriptions. Laboratory technicians receive lab test requests and update test results, while pharmacists manage prescription fulfillment and medication availability. Billing officers process billing updates and payment information, and hospital administrators use the system to request and review management reports that support monitoring of hospital activities.

Overall, the system serves as a centralized platform that supports communication and data exchange between patients, medical staff, administrative personnel, and hospital management.

3.7 Level-0 DFD

The Level-0 DFD decomposes the Hospital Database Management System into major processes.





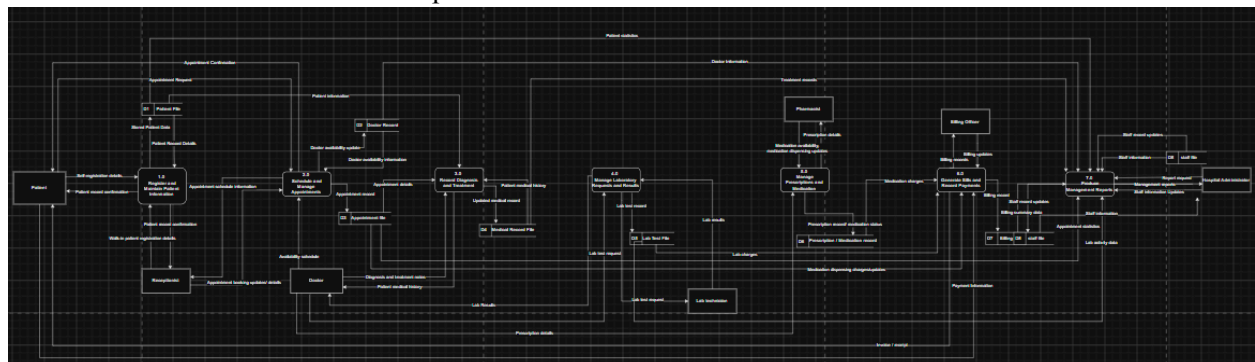
Link:

https://app.diagrams.net/#G1Sd4nBgmcyECA6Gm0sHLr0f8mP_dMdsz#/%7B%22pageId%22%3A%22KWXI2ywhoPq6-BKCYW6l%22%7D

The Level 0 DFD illustrates the main processes of the Hospital Database Management System and how data flows between external entities and internal data stores. Key processes include patient registration, appointment scheduling, diagnosis and treatment, laboratory management, prescription handling, billing, and report generation.

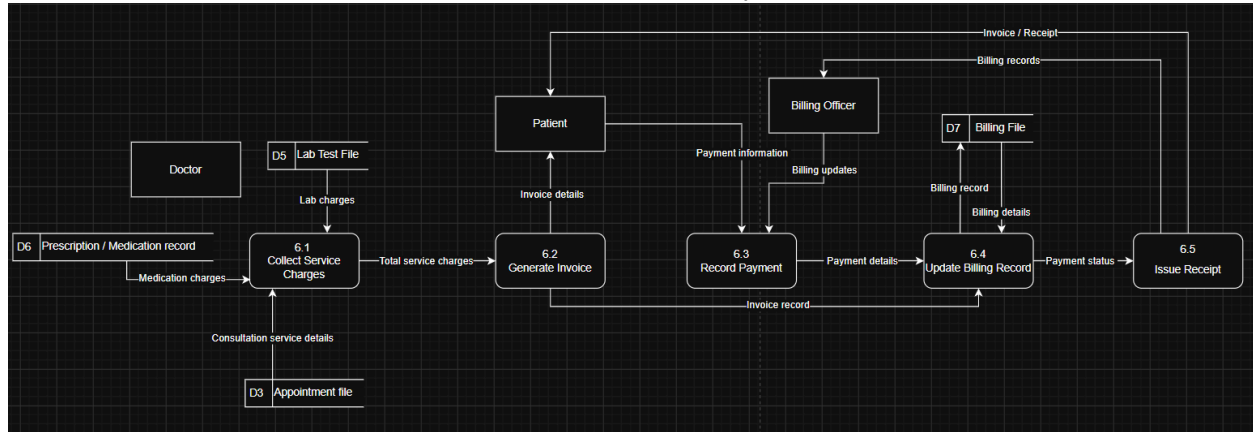
Patients and receptionists provide registration and appointment data, while doctors record diagnosis and treatment information. Laboratory technicians update test results, pharmacists manage prescriptions, and billing officers handle invoices and payments. The hospital administrator accesses system reports for monitoring and decision-making.

Overall, the diagram shows how data is processed, stored, and shared across different hospital functions to ensure efficient and coordinated operations.



3.8 Level-n DFDs

Level-1 DFD for Process 6.0: Generate Bills and Record Payments



Link:

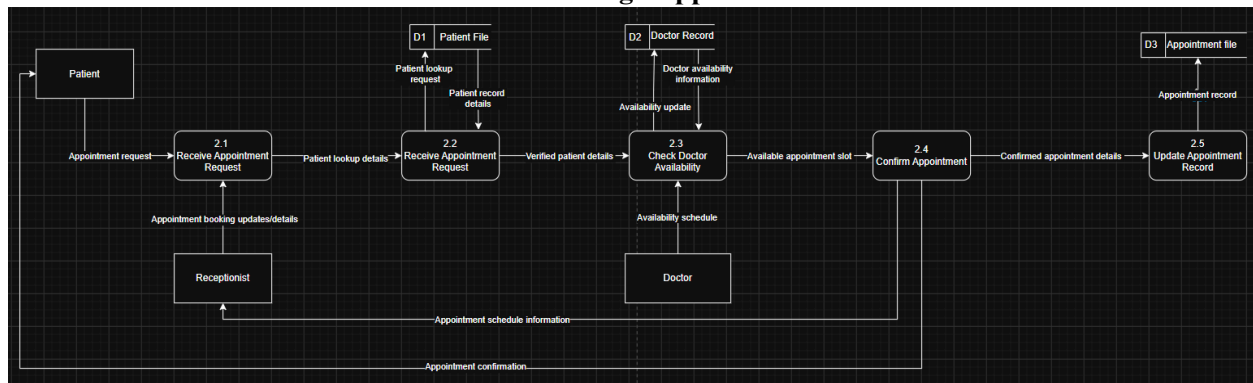
https://app.diagrams.net/#G1YcDTiwyhFhYhc2vIjCc8df5VsQ8kMdSm#/%7B%22pageId%22%3A%22E8jK4aLkOG_UL8VpNvvJ%22%7D

The Level-1 Data Flow Diagram for Process 6.0 describes how billing information is generated and payment transactions are processed within the system. The process begins by collecting service charge details from multiple data sources, including consultation services recorded in the Appointment File, laboratory services recorded in the Lab Test File, and medication charges stored in the Prescription/Medication File.

After gathering these service charges, the system generates an invoice that summarizes the total cost of services provided to the patient. Payment information is then received from the patient and billing updates may also be entered by the billing officer. The system records payment details and updates the Billing File to maintain accurate financial records.

Finally, the system generates a receipt for the patient and provides updated billing records to the billing officer for monitoring and verification. This decomposition demonstrates how the system integrates service charges from multiple departments to produce accurate invoices and maintain reliable payment tracking.

Level-1 DFD for Process 2.0: Schedule and Manage Appointments



Link:

https://app.diagrams.net/#G1YcDTiwyhFhYhc2vIjCc8df5VsQ8kMdSm#/%7B%22pageId%22%3A%22E8jK4aLkOG_UL8VpNvvJ%22%7D

The Level-1 Data Flow Diagram for Process 2.0 illustrates how appointment scheduling activities are handled within the Hospital Database Management System. The process begins when an appointment request is received from either the patient or the receptionist. The system then verifies the patient's information using the Patient File to ensure that the patient record exists and is valid before proceeding.

Next, the system checks doctor availability by referencing the Doctor File, which contains schedule and availability details. Based on this information, an appropriate appointment time slot is selected and confirmed. The confirmed appointment details are then stored in the Appointment File for future reference. Finally, appointment confirmation information is returned to the patient and schedule information is provided to the receptionist for coordination purposes.

This decomposition shows how the system ensures accurate scheduling by validating patient records, checking doctor availability, and maintaining appointment data in a centralized storage location.

3.9 Decision Tables

Decision tables are used to model business rules for selected DFD processes. In this project, we created decision tables for two Level-0 DFD processes:

- **Process 2.0: Schedule and Manage Appointments**
- **Process 6.0: Generate Bills and Record Payments**

The tables identify the conditions that affect each decision and the actions the system performs.

3.9.1 Decision Table 1: Appointment Scheduling

This decision table corresponds to Process 2.0: Schedule and Manage Appointments. It determines whether an appointment should be confirmed, rescheduled, or rejected.

3.9.1.1 Unsimplified Decision Table

Conditions	R1	R2	R3	R4	R5	R6	R7	R8
Patient registered?	Y	Y	Y	Y	N	N	N	N
Doctor Available?	Y	Y	N	N	Y	Y	N	N
Time slot free?	Y	N	Y	N	Y	N	Y	N
Actions								
Confirm appointment	✓							
Suggest an alternative slot		✓	✓	✓				
Reject request					✓	✓	✓	✓
Generate	✓							

appointment record								
--------------------	--	--	--	--	--	--	--	--

3.9.1.2 Appointment Scheduling Decision Table Reduction

Rule 1 (Old Rule 1):

- If the patient is registered, the doctor is available, and the time slot is free, the system confirms the appointment and generates an appointment record.
- This rule is kept separate because it is the only case where all scheduling conditions are satisfied.

Rule 2 (Old Rule 2):

- If the patient is registered and the doctor is available, but the time slot is not free, the system suggests an alternative time slot.
- This rule is kept separate because the doctor is available, but the selected time cannot be used.

Rule 3 (Old Rules 3 and 4):

- If the patient is registered but the doctor is not available, the system suggests an alternative appointment.
- These rules were combined because the outcome is the same regardless of whether the time slot is free or not. The condition “time slot free” is replaced with - (don’t care) because it does not affect the final action once the doctor is unavailable.

Rule 4 (Old Rules 5, 6, 7, and 8):

- If the patient is not registered, the system rejects the appointment request.
- These rules were combined because the outcome is the same regardless of doctor availability or time slot availability. The conditions are replaced with - (don’t care) since they do not affect the decision.

3.9.1.3 Simplified Decision Table

Conditions	R1	R2	R3	R4
Patient registered?	Y	Y	Y	N
Doctor Available?	Y	Y	N	-
Time slot free?	Y	N	-	-
Actions				
Confirm appointment	✓			
Suggest an alternative slot		✓	✓	

Reject request				✓
Generate appointment record	✓			

3.9.2 Decision Table 2: Billing and Payment Processing

This decision table defines the decision rules for how billing and payments are handled.

3.9.2.1 Unsimplified Decision Table

Conditions	R1	R2	R3	R4	R5	R6	R7	R8
Service completed?	Y	Y	Y	Y	N	N	N	N
Invoice generated?	Y	Y	N	N	Y	Y	N	N
Payment received?	Y	N	Y	N	Y	N	Y	N
Full payment?	Y	Y	N	N	Y	Y	N	N
Actions								
Generate invoice			✓	✓				
Record payment	✓		✓					
Update billing record	✓	✓	✓	✓				
Mark as pending		✓		✓	✓	✓	✓	✓
Issue receipt	✓							

3.9.2.2 Billing and Payment Decision Table Reduction

Rule 1 (Old Rules 3 and 4)

- If services are completed and no invoice has been generated, the system generates an invoice.
- These rules were combined because both cases result in the same action regardless of payment status.
The conditions “payment received” and “full payment” are replaced with - (don’t care) since they do not affect the outcome.

Rule 2 (Old Rules 2 and 6)

- If an invoice has been generated and no payment is received, the system updates the billing record and marks the bill as pending.
- These rules were combined because they lead to the same outcome.
The condition “full payment” is replaced with - (don’t care) since payment has not been made.

Rule 3 (Old Rule 1)

- If payment is received and the full amount is paid, the system records the payment, updates the billing record, and issues a receipt.
- This rule is kept separate because it represents a unique outcome where the billing process is completed.

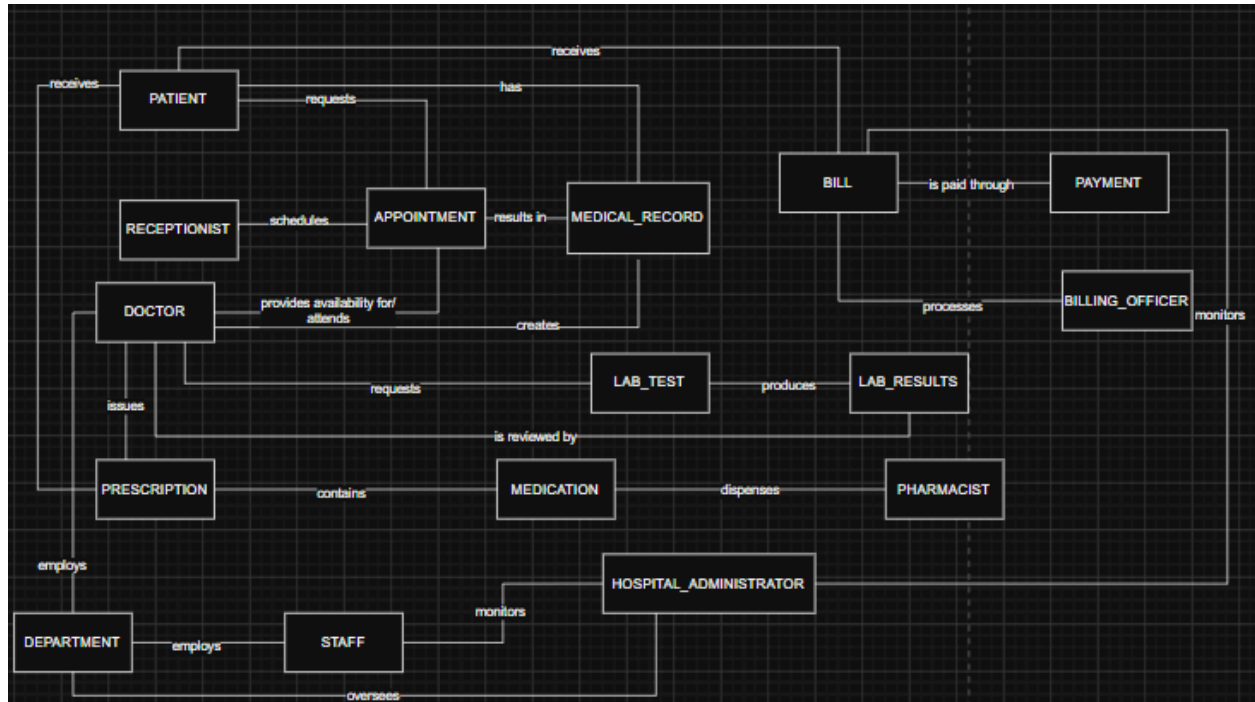
Rule 4 (Old Rules 5 and 7)

- If payment is received but the full amount is not paid, the system records the payment, updates the billing record, and marks the bill as pending.
- The condition “invoice generated” is replaced with - (don’t care) because it does not affect the outcome once payment is received.

3.9.2.3 Simplified Decision Table

Conditions	R1	R2	R3	R4
Service completed?	Y	Y	Y	Y
Invoice generated?	N	Y	Y	Y
Payment received?	-	N	Y	Y
Full payment?	-	-	Y	N
Actions				
Generate invoice	✓			
Record payment			✓	✓
Update billing record		✓	✓	✓
Mark as pending		✓		✓
Issue receipt			✓	

3.10 Abstract ERD



Link:

https://app.diagrams.net/#G1TwewU90l35Xs_U9l7G9Q1oU6zzGtyAff#%7B%22pageId%22%3A%22NAzgHVQulgW-D367ba_E%22%7D

The abstract Entity-Relationship Diagram (ERD) illustrates the main entities involved in the Hospital Database Management System and the relationships between them at a conceptual level. Patients request appointments that are scheduled by receptionists and attended by doctors, which result in medical records. Doctors may request lab tests that produce lab results used during diagnosis and treatment.

Doctors also issue prescriptions containing medications dispensed by pharmacists. Patients receive bills for hospital services, which are processed by billing officers and paid through payment transactions. Departments employ doctors and staff members who support hospital operations, while hospital administrators monitor staff activities and review system-generated reports.

This conceptual ERD provides a high-level overview of how clinical, administrative, and financial entities interact within the hospital system.

4. Phase 3: System Design

4.1 Designing the Human Interface

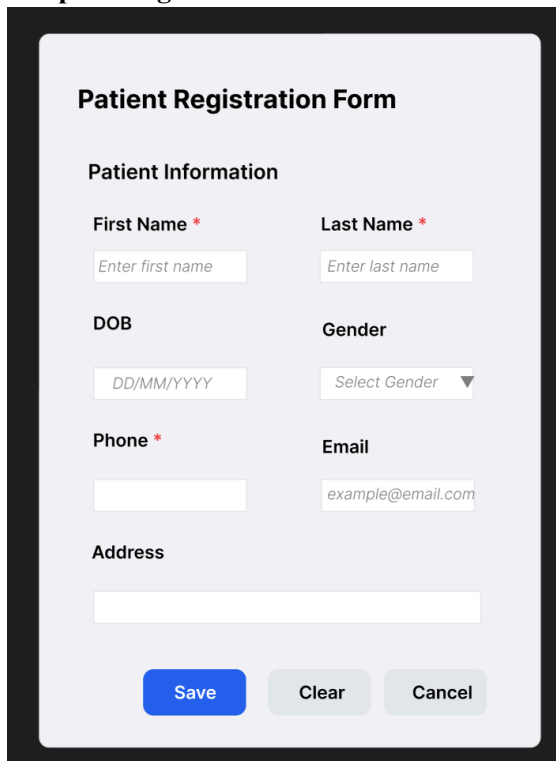
For the Hospital Database Management System, four interface designs were developed as a combination of forms and reports. The selected interfaces support the main system functions identified in Phase 2: patient registration, appointment scheduling, consultation documentation, and laboratory reporting.

4.1.1 Patient Registration Form

a. Narrative Overview

- Form: Patient Registration Form
- Users: Receptionists and administrative staff
- Tasks: Enter and manage patient information, including personal details and contact information
- System: Hospital Database Management System
- Environment: Hospital front desk using a desktop-based system

b. Sample Design



Patient Registration Form

Patient Information

First Name * **Last Name ***

DOB **Gender**

Phone * **Email**

Address

Figure 4.1: Patient Registration Form Interface

c. Testing and Usability Assessment

- Consistency (1 = consistent to 7 = inconsistent): 1.4
- Efficiency (1 = efficient to 7 = inefficient): 1.5
- Accuracy (1 = accurate to 7 = inaccurate): 1.3
- Ease of Use (1 = easy to 7 = difficult): 1.2

The interface is easy to use due to its clear structure and logical grouping of fields. Required fields are indicated, and input fields are aligned consistently, reducing user confusion and improving efficiency.

This evaluation is based on heuristic usability principles rather than formal user testing.

d. Input Error Handling

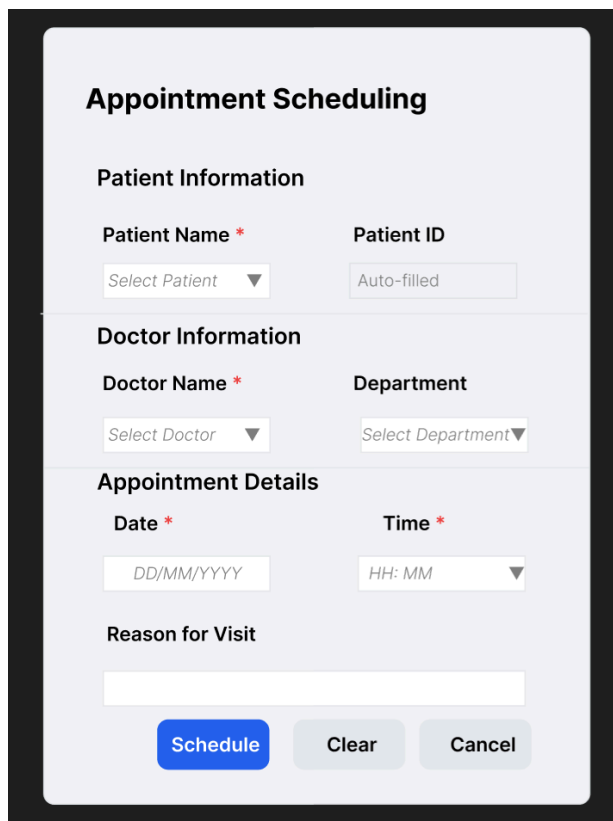
- Required fields must not be left empty
- Phone number must follow a valid format
- Email must be in a valid format
- Date of birth cannot be in the future
- Clear error messages are displayed when invalid input is entered

4.1.2 Appointment Scheduling Form

a. Narrative Overview

- Form: Appointment Scheduling Form
- Users: Receptionists
- Tasks: Schedule appointments based on patient selection, doctor availability, and time slots
- System: Hospital Database Management System
- Environment: Hospital scheduling system accessed via desktop interface
-

b. Sample Design



The image shows a sample design of an appointment scheduling interface. It is a light purple form with a dark purple border. The form is titled "Appointment Scheduling" in bold black text. Below the title, there are three sections: "Patient Information", "Doctor Information", and "Appointment Details".

Patient Information

Patient Name * (dropdown menu with "Select Patient" text) Patient ID (text input field with "Auto-filled" text)

Doctor Information

Doctor Name * (dropdown menu with "Select Doctor" text) Department (dropdown menu with "Select Department" text)

Appointment Details

Date * (text input field with "DD/MM/YYYY" text) Time * (dropdown menu with "HH: MM" text)

Reason for Visit (text input field)

At the bottom of the form, there are three buttons: "Schedule" (blue), "Clear" (light blue), and "Cancel" (light blue).

Figure 4.2: Appointment Scheduling Interface

c. Testing and Usability Assessment

- Consistency: 1.5
- Efficiency: 1.6
- Accuracy: 1.4
- Ease of Use: 1.3

The interface provides a clear workflow for scheduling appointments. Dropdown selections reduce input errors, while the structured grouping of patient, doctor, and appointment details improves clarity and usability.

d. Input Error Handling

- Patient must be selected before scheduling
- Doctor availability must be verified
- Date cannot be in the past
- Time must be selected from available slots
- Required fields must be completed before submission

4.1.3 Doctor Consultation Form

a. Narrative Overview

- Form: Doctor Consultation Form
- Users: Doctors
- Tasks: View patient information and medical history, record diagnosis and treatment details, and issue prescriptions for medications.
- System: Hospital Database Management System
- Environment: Clinical workstation used by doctors during patient consultations within the hospital.

b. Sample Design

Figure 4.3: Doctor Consultation Interface

c. Testing and Usability Assessment

Consistency (1 = consistent to 7 = inconsistent): 1.4

Efficiency (1 = efficient to 7 = inefficient): 1.5

Accuracy (1 = accurate to 7 = inaccurate): 1.3

Ease of Use (1 = easy to 7 = difficult): 1.2

The interface is designed to support efficient clinical workflows by organizing information into clearly defined sections. The use of multi-line input fields for medical history and treatment notes allows detailed documentation, while dropdown selections for patient and medication reduce input errors. Consistent layout and alignment improve readability and usability.

d. Input Error Handling

- Patient must be selected before recording consultation details
- The diagnosis field must not be left empty
- Medication must be selected before saving prescription
- Dosage must follow a valid format (e.g., numeric value with unit)
- Medical history and treatment notes fields accept only valid text input
- Clear error messages are displayed when required fields are missing or invalid

4.1.4 Laboratory Results Report

a. Narrative Overview

- Form: Laboratory Results Report
- Users: Doctors and laboratory technicians
- Tasks: View laboratory test requests, monitor test status, and review patient test results to support diagnosis and treatment decisions.
- System: Hospital Database Management System
- Environment: Clinical and laboratory workstations within the hospital, accessed through a desktop-based system.

b. Sample Design

The screenshot displays a web-based interface for viewing laboratory results. At the top, the title 'Laboratory Results Report' is centered. Below it, there are input fields for 'Patient' (with a placeholder 'Patient Name'), 'ID' (with a placeholder '#####'), 'Doctor' (with a placeholder 'Doctor Name'), and 'Dept' (with a placeholder 'Cardiology'). A 'Date Range' section includes two date pickers with placeholders 'DD/MM/YYYY' and a minus sign between them. Below these fields is a table with six columns: 'Test Name', 'Requested Date', 'Status', 'Results', 'Units', and 'Ref Range'. The table contains three rows of data: 'CBC' with a status of 'Done', 'Glucose' with a status of 'Done', and 'CRP' with a status of 'Pending'. At the bottom of the interface, there is a 'Notes' section with a text area for 'Enter additional notes or observations' and three buttons: 'Print', 'Export', and 'Close'.

Test Name	Requested Date	Status	Results	Units	Ref Range
CBC	12/05/2026	Done	5.4	x10 ⁹ /L	4.0-11.0
Glucose	12/05/2026	Done	98	mg/dL	70-99
CRP	13/05/2026	Pending	-	-	-

Figure 4.4: Laboratory Results Report Interface

c. Testing and Usability Assessment

- Consistency (1 = consistent to 7 = inconsistent): 1.4
- Efficiency (1 = efficient to 7 = inefficient): 1.5
- Accuracy (1 = accurate to 7 = inaccurate): 1.3
- Ease of Use (1 = easy to 7 = difficult): 1.2

The report presents information in a structured tabular format, allowing users to quickly scan and interpret laboratory results. Clear column headings and consistent alignment improve readability, while color-coded status indicators help distinguish completed and pending tests. The layout ensures that patient details and test results are logically grouped, supporting efficient clinical decision-making.

d. Input Error Handling

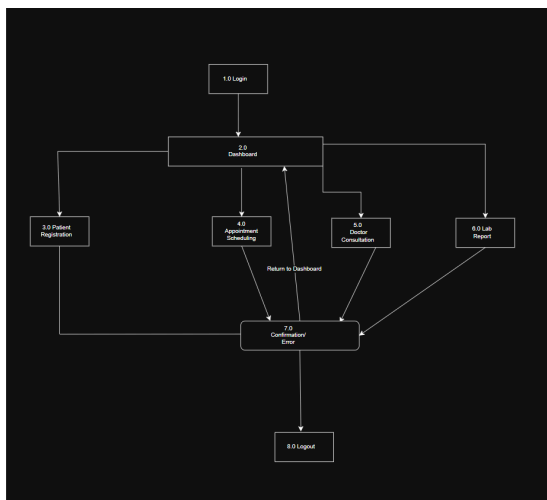
- Invalid or missing patient selection displays an appropriate message
- Date range must be valid, with the start date occurring before the end date
- If no test records are available, the system displays “No records found.”
- Incomplete laboratory tests are clearly indicated as “Pending.”
- Missing or unavailable values are represented using “-” to avoid misinterpretation

N.B. The evaluation for Testing and Usability Assessment is based on heuristic usability principles rather than formal user testing.

4.2 Interface Dialogue Design

4.2.1 Dialogue Diagram

The dialogue diagram illustrates the navigation flow of the system. Users log in and access a role-based dashboard, from which they navigate to core interfaces. After completing actions, the system provides confirmation or error feedback and returns users to the dashboard. The design ensures consistency, feedback, user control, and ease of navigation.



https://drive.google.com/file/d/1U_Zt79s2d3fLpORyG7Yf9ZvpB7eCUNs/view?usp=sharing

Figure 4.5: Interface Dialogue Diagram

4.3 Anticipating and Handling Input Errors

The system anticipates input errors using standard data validation techniques and structured data entry design. These include checks for missing data, format validation, range validation, and controlled input values such as dropdown menus. These techniques are based on system design guidelines for detecting and preventing common data entry errors.

Validation Type	Error
Missing Data	Required fields (name, phone, diagnosis) must be filled out

Format	Email must follow a valid format
Range	Date of birth cannot be in the future
Values(Lookup)	Gender, doctor, department selected from dropdown
Reasonableness	Appointment time must match available slots

Error Message Handling

Clear and specific error messages are used to guide users in correcting invalid inputs.

Examples:

- “Please enter a valid email address.”
- “Required fields must be completed.”
- “Date cannot be in the future.”

4.4 Logical Design

4.4.1 Entity and Attribute Identification

The logical model of the Hospital Database Management System is developed by identifying the main entities and their associated attributes. Each entity represents a key component of the system, and attributes describe the properties of each entity. Primary keys uniquely identify each record, while foreign keys establish relationships between entities.

Patient Entity

Attribute	Type	Logical Data Type	Domain / Business Restriction
PatientId	Primary Key	Integer	Unique, not null
FirstName	Simple	Text	Required, max 50 characters
LastName	Simple	Text	Required, max 50 characters
DOB	Simple	Date	Cannot be in the future
Gender	Simple	Text	Must be Male, Female, or Other
Phone	Simple	Text	Required, valid phone format
Email	Simple	Text	Valid email format
Address	Simple	Text	Includes street/building details

City	Simple	Text	Required

Doctor Entity

Attribute	Type	Logical Data Type	Domain / Business Restriction
DoctorId	Primary Key	Integer	Unique, not null
Name	Simple	Text	Required, max 100 characters
Department	Simple	Text	Required, e.g., Cardiology, Pediatrics, Internal Medicine

Appointment Entity

Attribute	Type	Logical Data Type	Domain / Business Restriction
AppointmentId	Primary Key	Integer	Unique, not null
PatientId	Foreign Key	Integer	References Patient
DoctorId	Foreign Key	Integer	References Doctor
AppointmentDate	Simple	Date	Required, can not be in the past
AppointmentTime	Simple	Time	Required, must be within working hours
ReasonForVisit	Simple	Text	Optional, max 255 characters
AppointmentStatus	Simple	Text	Must be Scheduled, Completed, Canceled, or Rescheduled

Consultation Entity

Attribute	Type	Logical Data Type	Domain / Business Restriction
ConsultationId	Primary Key	Integer	Unique, not null
PatientId	Foreign Key	Integer	References Patient
DoctorId	Foreign Key	Integer	References Doctor

AppointmentId	Foreign Key	Integer	References Appointment
ConsultationDate	Simple	Date	Required, can not be in the future
MedicalHistory	Simple	Long Text	Optional
Diagnosis	Simple	Long Text	Optional
TreatmentNotes	Simple	Long Text	Optional

Prescription Entity

Attribute	Type	Logical Data Type	Domain / Business Restriction
PrescriptionId	Primary Key	Integer	Unique, not null
ConsultationIdId	Foreign Key	Integer	References Consultation
MedicationName	Simple	Text	Required
Dosage	Simple	Text	Required, e.g., 500 mg twice daily
PrescriptionDate	Simple	Date	Required, cannot be in the future
PrescriptionStatus	Simple	Text	Must be Active, Completed, or Cancelled

LabTest Entity

Attribute	Type	Logical Data Type	Domain / Business Restriction
LabTestId	Primary Key	Integer	Unique, not null
PatientId	Foreign Key	Integer	References Patient
DoctorId	Simple	Integer	References Doctor
TestName	Simple	Text	Required
RequestedDate	Simple	Date	Required
ResultDate	Simple	Date	Optional, cannot be before RequestedDate
ResultValue	Simple	Text	Optional until completed
Units	Simple	Text	Optional, e.g., mg/dL, x10 ⁹ /L
ReferenceRange	Simple	Text	Optional

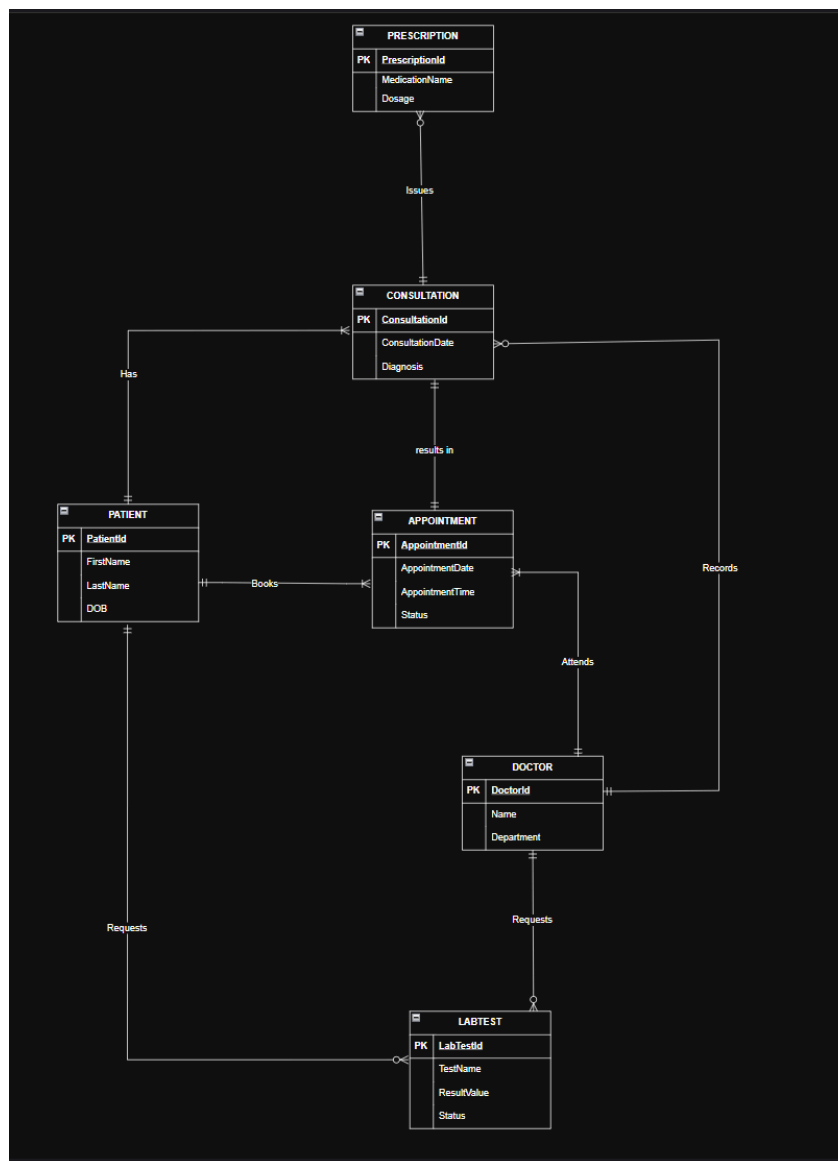
Status	Simple	Text	Must be Pending, Completed, or Cancelled
--------	--------	------	--

4.4.2 Relationship Cardinalities and Labels

The relationships between entities are illustrated using an Entity-Relationship Diagram (ERD). The diagram shows the main entities, their key attributes, and the relationships between them, including cardinalities and descriptive labels.

a. Entity-Relationship Diagram

https://drive.google.com/file/d/19OiiUeT_SoCs5vTVG3HBIZhwYymbZvXd/view?usp=sharing



Relationship Label	Entity 1	Entity 2	Cardinality	Explanation
books	Patient	Appointment	1:M	One patient can book many appointments; each appointment belongs to one patient
attends	Doctor	Appointment	1:M	One doctor can attend many appointments; each appointment is assigned to one doctor
Results in	Appointment	Consultation	1:1	Each appointment results in one consultation record
has	Patient	Consultation	1:M	One patient can have multiple consultation records
records	Doctor	Consultation	1:M	One doctor can record multiple consultations
issues	Consultation	Prescription	1:M	One consultation can issue multiple prescriptions
requests	Doctor	LabTest	1:M	One doctor can request multiple lab tests
undergoes	Patient	LabTest	1:M	One patient can undergo multiple lab tests

4.4.3 Relational Model / Shorthand Relations

- PATIENT(PatientID PK, FirstName, LastName, DOB, Gender, Phone, Email, Address, City)
- DOCTOR(DoctorID PK, Name, Department)
- APPOINTMENT(AppointmentID PK, PatientID FK, DoctorID FK, AppointmentDate, AppointmentTime, ReasonForVisit, AppointmentStatus)
- CONSULTATION(ConsultationID PK, PatientID FK, DoctorID FK, AppointmentID FK, ConsultationDate, MedicalHistory, Diagnosis, TreatmentNotes)

- PRESCRIPTION(PrescriptionID PK, ConsultationID FK, MedicationName, Dosage, PrescriptionDate, PrescriptionStatus)
- LABTEST(LabTestID PK, PatientID FK, DoctorID FK, TestName, RequestedDate, ResultDate, ResultValue, Units, ReferenceRange, Status)

4.4.4 Normalization to 3NF

Unnormalized Example

→ APPOINTMENT_FULL(AppointmentID, PatientName, PatientPhone, DoctorName, Department, AppointmentDate, AppointmentTime, Diagnosis, MedicationName, Dosage)

This unnormalized table combines patient, doctor, appointment, consultation, and prescription information in one relation. This can cause repeated patient and doctor information and makes updates difficult.

Step 1: First Normal Form (1NF)

To satisfy 1NF, repeating groups are removed, and values are made atomic. Since one appointment may include more than one medication, medication details are separated.

APPOINTMENT_FULL(AppointmentID, PatientName, PatientPhone, DoctorName, Department, AppointmentDate, AppointmentTime, Diagnosis)

PRESCRIPTION_TEMP(AppointmentID, MedicationName, Dosage)

Step 2: Second Normal Form (2NF)

To satisfy 2NF, attributes must depend on the full primary key. Patient and doctor details are separated into their own relations.

PATIENT(PatientID PK, FirstName, LastName, Phone)

DOCTOR(DoctorID PK, Name, Department)

APPOINTMENT(AppointmentID PK, PatientID, DoctorID, AppointmentDate, AppointmentTime)

This removes partial dependencies by ensuring that attributes such as patient and doctor details depend only on their respective primary keys.

Step 3: Third Normal Form (3NF)

To satisfy 3NF, transitive dependencies are removed. Consultation and prescription details are separated so that each relation stores data about one main concept.

CONSULTATION(ConsultationID PK, AppointmentID FK, Diagnosis)

PRESCRIPTION(PrescriptionID PK, ConsultationID FK, MedicationName, Dosage)

Final Normalized Relations

PATIENT(PatientID PK, FirstName, LastName, DOB, Gender, Phone, Email, Address, City)

DOCTOR(DoctorID PK, Name, Department)

APPOINTMENT(AppointmentID PK, PatientID FK, DoctorID FK, AppointmentDate, AppointmentTime, ReasonForVisit, AppointmentStatus)

CONSULTATION(ConsultationID PK, PatientID FK, DoctorID FK, AppointmentID FK, ConsultationDate, MedicalHistory, Diagnosis, TreatmentNotes)

PRESCRIPTION(PrescriptionID PK, ConsultationID FK, MedicationName, Dosage, PrescriptionDate, PrescriptionStatus)

LABTEST(LabTestID PK, PatientID FK, DoctorID FK, TestName, RequestedDate, ResultDate, ResultValue, Units, ReferenceRange, Status)

→ The final relations are in 3NF because all attributes are atomic, each non-key attribute depends on the primary key, and no non-key attribute depends on another non-key attribute. Data is separated into distinct entities such as Patient, Doctor, Appointment, Consultation, Prescription, and LabTest to eliminate redundancy and prevent update, insertion, and deletion anomalies.

4.5 Physical Design Modeling

4.5.1 DBMS Selection

The database was implemented using Oracle Database through SQL Developer. We selected it because we are familiar with it from other assignments and due to its support for relational database design, enforcement of constraints, and efficient data manipulation using SQL.

4.5.2 Table Creation and Data Types

The database tables were created using SQL CREATE TABLE statements. Appropriate Oracle data types were used, such as:

- NUMBER for identifiers and keys
- VARCHAR2 for textual data
- DATE for date attributes

Each table includes a primary key, and relevant attributes were defined with NOT NULL, UNIQUE, and CHECK constraints to enforce data integrity.

```

CREATE TABLE Doctor (
    DoctorID NUMBER PRIMARY KEY,
    DoctorName VARCHAR2(100) NOT NULL,
    Department VARCHAR2(50) NOT NULL
);

CREATE TABLE Patient (
    PatientID NUMBER PRIMARY KEY,
    FirstName VARCHAR2(50) NOT NULL,
    LastName VARCHAR2(50) NOT NULL,
    DOB DATE NOT NULL,
    Gender VARCHAR2(10) CHECK (Gender IN ('Male', 'Female', 'Other')),
    Phone VARCHAR2(20) NOT NULL,
    Email VARCHAR2(100) UNIQUE,
    Address VARCHAR2(150),
    City VARCHAR2(50)
);

```

```

CREATE TABLE Appointment (
    AppointmentID NUMBER PRIMARY KEY,
    PatientID NUMBER NOT NULL,
    DoctorID NUMBER NOT NULL,
    AppointmentDate DATE NOT NULL,
    AppointmentTime VARCHAR2(10) NOT NULL,
    ReasonForVisit VARCHAR2(255),
    AppointmentStatus VARCHAR2(20) DEFAULT 'Scheduled'
    CHECK (AppointmentStatus IN ('Scheduled', 'Completed', 'Cancelled', 'Rescheduled')),
    CONSTRAINT fk_app_patient FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    CONSTRAINT fk_app_doctor FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

CREATE TABLE Consultation (
    ConsultationID NUMBER PRIMARY KEY,
    PatientID NUMBER NOT NULL,
    DoctorID NUMBER NOT NULL,
    AppointmentID NUMBER NOT NULL,
    ConsultationDate DATE NOT NULL,
    Medication VARCHAR2(100),
    Diagnosis VARCHAR2(100) NOT NULL,
    TreatmentNotes VARCHAR2(500),
    CONSTRAINT fk_cons_patient FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    CONSTRAINT fk_cons_doctor FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID),
    CONSTRAINT fk_cons_appointment FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID)
);

```

4.5.3 Constraints and Relationships

Primary key and foreign key constraints were implemented to maintain referential integrity between tables. For example:

- PatientID in Appointment references the Patient table
- DoctorID in multiple tables ensures correct association with doctors
- ConsultationID links prescriptions to consultations

Additional constraints, such as CHECK were used to restrict valid values (e.g., gender, status fields).

4.5.4 Indexes

Indexes were created on selected non-key attributes to improve query performance. These include:

- Patient's last name
- Doctor department
- Appointment date
- Consultation date
- Medication name
- Lab test status

Indexes allow faster data retrieval, especially for search and filtering operations.

```

CREATE INDEX idx_patient_name ON Patient(LastName);
CREATE INDEX idx_doctor_department ON Doctor(Department);
CREATE INDEX idx_appointment_date ON Appointment(AppointmentDate);
CREATE INDEX idx_consultation_date ON Consultation(ConsultationDate);
CREATE INDEX idx_prescription_medication ON Prescription(MedicationName);
CREATE INDEX idx_labtest_status ON LabTest(Status);

INSERT INTO Patient VALUES (1, 'Aisha', 'Rhan', TO_DATE('2002-04-12','YYYY-MM-DD'), 'Fema
INSERT INTO Patient VALUES (2, 'Omar', 'Ali', TO_DATE('1998-09-20','YYYY-MM-DD'), 'Male',

```

Script Output x Query Result x

Task completed in 0.168 seconds

Index IDX_PATIENT_NAME created.

Index IDX_DOCTOR_DEPARTMENT created.

Index IDX_APPOINTMENT_DATE created.

Index IDX_CONSULTATION_DATE created.

Index IDX_PRESCRIPTION_MEDICATION created.

Index IDX_LABTEST_STATUS created.

4.5.5 Sample Data and Implementation

Sample records were inserted into all tables using INSERT INTO statements. These records demonstrate how patient, doctor, appointment, consultation, prescription, and lab test data are stored and related.

SELECT * FROM table_name was executed to verify that data was correctly inserted and relationships were properly maintained.

Below, we included the output of the LabTest table. We will show the output of the other tables more in the video.

```

SELECT * FROM LabTest;

SELECT table_name
FROM user_tables
ORDER BY table_name;

```

Script Output x Query Result x

All Rows Fetched: 5 in 0.143 seconds

LABTESTID	PATIENTID	DOCTORID	TESTNAME	REQUESTEDDATE	RESULTDATE	RESULTVALUE	UNITS	REFERENCERANGE	STATUS
1	1	2	2 Blood Test	11-MAY-26	12-MAY-26	Normal	mg/dL	70-99	Completed
2	2	1	1 ECG	10-MAY-26	(null)	(null)	(null)	(null)	Pending
3	3	3	3 CBC	12-MAY-26	(null)	(null)	(null)	(null)	Pending
4	4	4	4 Allergy Test	13-MAY-26	14-MAY-26	Positive	(null)	Negative	Completed
5	5	1	2 Vitamin B12 Test	14-MAY-26	15-MAY-26	Low	pg/mL	200-900	Completed

4.5.6 System Demonstration

The following queries demonstrate the created tables and retrieval of data.

→SELECT table_name FROM user_tables ORDER BY table_name;

Script Output x Query Result 12 x

All Rows Fetched: 6 in 0.269 second

TABLE_NAME
1 APPOINTMENT
2 CONSULTATION
3 DOCTOR
4 LABTEST
5 PATIENT
6 PRESCRIPTION

→To demonstrate that relationships between tables are correctly implemented, a join query was executed to retrieve combined data from multiple tables.

Script Output x Query Result x

All Rows Fetched: 5 in 0.165 seconds

FIRSTNAME	LASTNAME	DOCTORNAME	APPOINTMENTDATE	APPOINTMENTSTATUS
1 Aisha	Khan	Dr. Sara Ahmed	10-MAY-26	Scheduled
2 Omar	Ali	Dr. Khalid Noor	11-MAY-26	Completed
3 Maya	Hassan	Dr. Lina Faris	12-MAY-26	Scheduled
4 Yousef	Nasser	Dr. Ahmed Mansour	13-MAY-26	Completed
5 Aisha	Khan	Dr. Khalid Noor	14-MAY-26	Rescheduled

→ SELECT * FROM Patient;

Script Output x Query Result x

All Rows Fetched: 4 in 0.146 seconds

PATIENTID	FIRSTNAME	LASTNAME	DOB	GENDER	PHONE	EMAIL	ADDRESS	CITY
1	Aisha	Khan	12-APR-02	Female	0501234567	aisha@example.com	Dubai Silicon Oasis	Dubai
2	Omar	Ali	20-SEP-98	Male	0559876543	omar@example.com	Mirdif	Dubai
3	Maya	Hassan	15-JAN-05	Female	0523334455	maya@example.com	Jumeirah	Dubai
4	Yousef	Nasser	03-JUL-87	Male	0562227788	yousef@example.com	Al Nahda	Dubai

5. Phase 4: Web Application Implementation

5.1 Application Overview

We used Oracle APEX to implement a web-based Hospital Management System and connect the designed user interfaces with the relational database. The system allows users to insert, update, and view hospital data through interactive forms and reports

5.2 Login Interface

The login page provides secure access to the system by allowing authorized users to enter credentials before accessing application modules. This ensures controlled access to patient, appointment, and billing information.

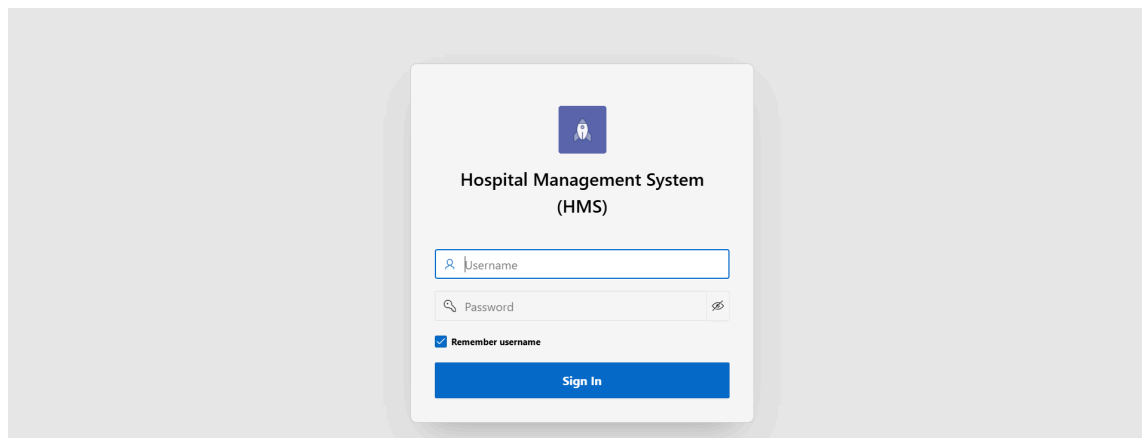


Figure 5.1: Login Page

5.3 Dashboard Interface

The homepage was designed as a system dashboard that provides quick navigation to major hospital modules such as patient registration, appointment scheduling, billing processing, and reports. This improves usability by organizing system functions in a centralized interface.

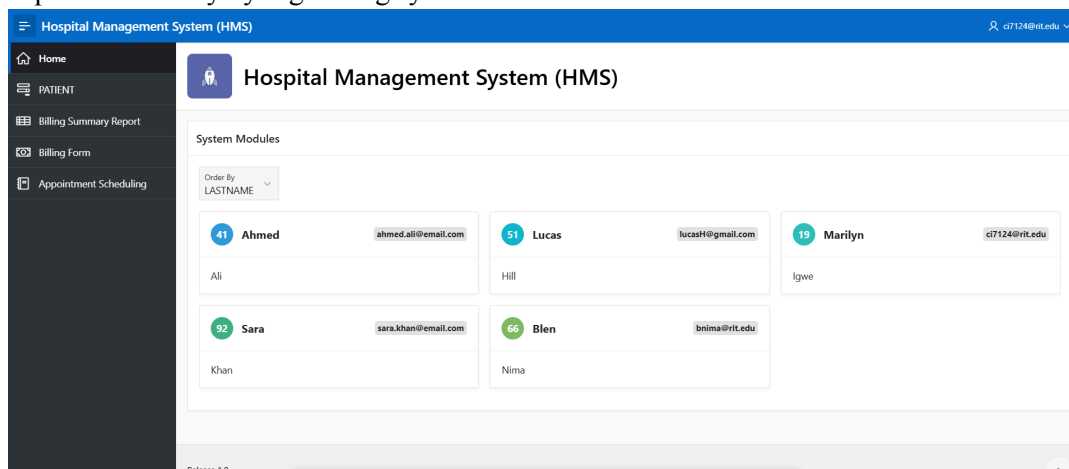
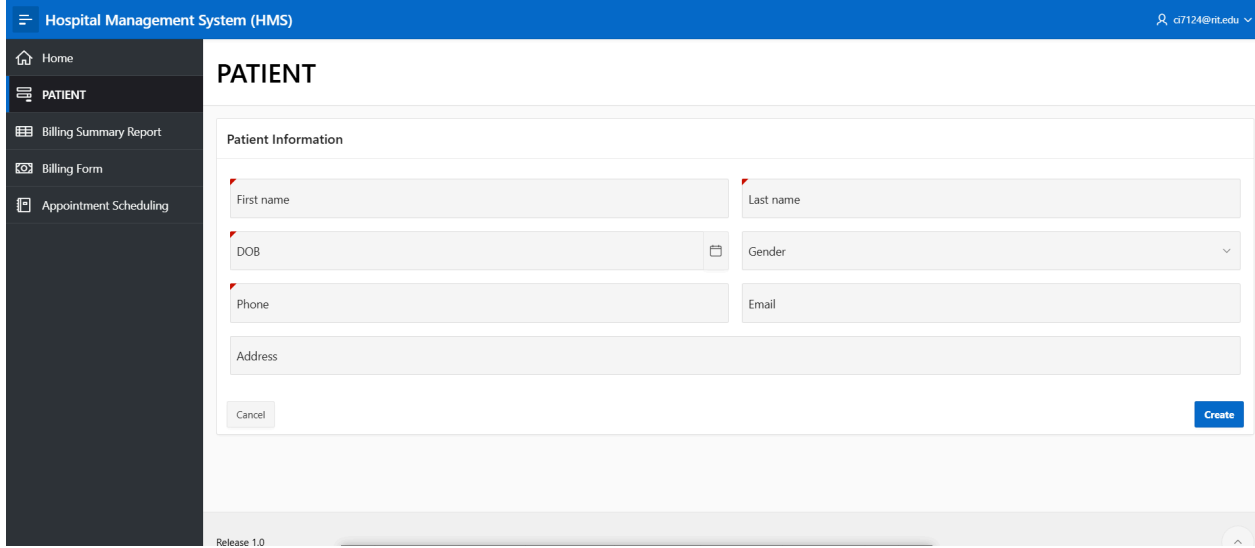


Figure 5.2: System Modules Dashboard

5.4 Patient Registration Form

The Patient Registration form allows users to add and manage patient records. Data entered through the interface is stored directly in the PATIENT table and becomes immediately available for retrieval and reporting.

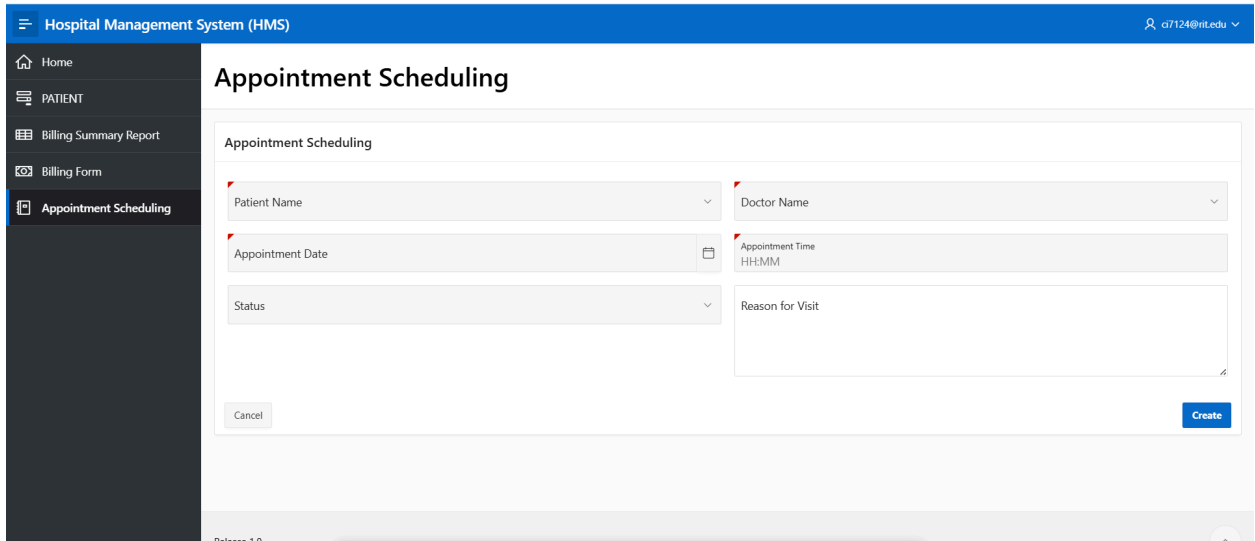


The screenshot shows the 'PATIENT' registration form within the HMS interface. The left sidebar contains navigation links: Home, PATIENT, Billing Summary Report, Billing Form, and Appointment Scheduling. The main content area is titled 'PATIENT' and contains a 'Patient Information' form. The form fields are: First name, Last name, DOB (with a calendar icon), Gender (dropdown), Phone, Email, and Address. There are 'Cancel' and 'Create' buttons at the bottom right. The footer indicates 'Release 1.0'.

Figure 5.3: Patient Registration Form

5.5 Appointment Scheduling Form

The Appointment Scheduling form enables users to create and manage patient appointments. Appointment details are stored in the APPOINTMENT table and linked to both patient and doctor records.



The screenshot shows the 'Appointment Scheduling' form within the HMS interface. The left sidebar contains navigation links: Home, PATIENT, Billing Summary Report, Billing Form, and Appointment Scheduling. The main content area is titled 'Appointment Scheduling' and contains an 'Appointment Scheduling' form. The form fields are: Patient Name (dropdown), Doctor Name (dropdown), Appointment Date (with a calendar icon), Appointment Time (HH:MM), Status (dropdown), and Reason for Visit (text area). There are 'Cancel' and 'Create' buttons at the bottom right. The footer indicates 'Release 1.0'.

Figure 5.4: Appointment Scheduling Form

5.6 Billing Form

The Billing Form allows billing officers to generate invoices associated with patient visits. Billing data is stored in the BILL table and supports financial tracking within the system.

The screenshot shows the 'Billing Form' interface within the 'Hospital Management System (HMS)'. The left sidebar contains navigation links: Home, PATIENT, Billing Summary Report, Billing Form (selected), and Appointment Scheduling. The main content area is titled 'Billing Form' and contains a form with the following fields: Patient Name (dropdown), Appointment Date (dropdown), Bill Date (dropdown with a calendar icon), Total Amount (text input), and Status (dropdown). At the bottom left is a 'Cancel' button, and at the bottom right is a 'Create' button.

Figure 5.5: Billing Form

5.7 Billing Summary Report

The Billing Summary Report displays stored billing records using an interactive report interface. Users can search, filter, and review invoice information dynamically.

The screenshot shows the 'Billing Summary Report' interface within the 'Hospital Management System (HMS)'. The left sidebar contains navigation links: Home, PATIENT, Billing Summary Report (selected), Billing Form, and Appointment Scheduling. The main content area is titled 'Billing Summary Report' and contains a table with the following data:

Patient Name	Appointment Time	Payment Date	Total Amount	Payment Status
Marilyn	10:30	5/30/2026	50000	Paid
Ahmed	10:30	5/10/2026	335	Partially Paid
Sara	10:30	5/15/2026	2500	Partially Paid
Blen	10:30	5/19/2026	40666	Partially Paid

At the top left of the table area is a search bar with a magnifying glass icon and a 'Go' button. At the top right is an 'Actions' dropdown menu. At the bottom right of the table is a pagination link '1 - 4'.

Figure 5.6: Billing Summary Report

5.8 Database Connectivity

The Oracle APEX application communicates directly with the implemented database schema. Form submissions insert records into database tables, while report pages retrieve stored data dynamically. This confirms successful integration between the user interface and the relational database.

6. Conclusion

Our project presented the design and implementation of a Hospital Database Management System aimed at improving the efficiency, accuracy, and organization of hospital operations. Through the application of system development methodologies, the project progressed from identifying existing problems to proposing and developing a structured database solution.

In Phase 1, the need for a centralized system was established, highlighting inefficiencies in manual and fragmented data management processes. Phase 2 focused on analyzing system requirements using techniques such as interviews, Data Flow Diagrams, decision tables, and an abstract ERD to model system behavior and data relationships. Phase 3 translated these requirements into a detailed system design, including user interfaces, logical data modeling, normalization, and physical database implementation using SQL.

Finally, Phase 4 demonstrated the practical implementation of the system through a web-based application developed in Oracle APEX. The application successfully connected user interfaces to the underlying database, allowing real-time data entry and retrieval through forms and reports.

Overall, the project achieved its objective of developing a functional and well-structured hospital database system. It demonstrates how database design and system analysis techniques can be applied to solve real-world problems, improve data management, and support efficient decision-making in a healthcare environment.

7. References

- [1] Valacich, J. S., & George, J. F. (2020). *Modern Systems Analysis and Design* (9th ed.). Pearson Education.
- [2] Oracle Corporation. (2024). Oracle APEX Application Development. Retrieved from <https://apex.oracle.com/en/>
- [3] Oracle Corporation. (2024). Oracle SQL Developer Documentation. Retrieved from <https://www.oracle.com/database/sqldeveloper/>
- [4] draw.io (Diagrams.net). (2024). Diagramming and visual communication tool. Retrieved from <https://app.diagrams.net>
- [5] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- [6] Date, C. J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
- [7] World Health Organization. (2019). *Classification of digital interventions, services and applications in health*. WHO Press.